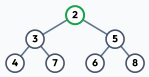
HEAP Priority Queue (FAANG Cheat Sheet)

1. WHAT IS A HEAP?

- A Heap is a **Complete Binary Tree**.
- It satisfies the **Heap Property**.
- It is **NOT** a sorted data structure.
- Only the root element is guaranteed to be the minimum (Min Heap) or maximum (Max Heap).
- Heaps are used to implement **Priority Queue**.

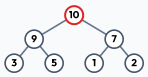
2. MIN HEAP vs MAX HEAP

MIN HEAP
Parent ≤ Children



Smallest element is at the root.

MAX HEAP
Parent ≥ Children



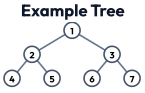
Largest element is at the root.

3. COMPLETE BINARY TREE

All levels are completely filled except possibly the last level, and all nodes in the last level are as far left as possible.

4. ARRAY REPRESENTATION (0-INDEXED)

Example Tree



Index Relationships
Parent(i) = (i - 1) / 2
Left(i) = 2 * i + 1
Right(i) = 2 * i + 2

Idx:	0	1	2	3	4	5	6
Val:	1	2	3	4	5	6	7

5. TIME COMPLEXITIES

Operation	Min Heap	Max Heap	Why?
offer(x)	O(log n)	O(log n)	Heapify Up
poll()	O(log n)	O(log n)	Heapify Down
peek()	O(1)	O(1)	Root access
size()	O(1)	O(1)	Just length
Build Heap	O(n)	O(n)	Bottom-up heapify

Build Heap (Bottom-Up) is O(n) which is better than inserting n elements one by one (O(n log n)).

6. JAVA PRIORITYQUEUE

a) Min Heap (Default)

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
```

b) Max Heap (3 Ways)

1. Collections.reverseOrder() (Recommended)

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOr
```

2. Using Comparator (Integer.compare)

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> Integer.com
```

3. Using subtraction (Not recommended)

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
```

c) Common Operations

Method	Description
`offer(x)`	Inserts element x into heap.
`poll()`	Removes and returns root element (min or max).
`peek()`	Returns root element without removing.
`size()`	Returns number of elements.

7. IMPORTANT TERMINOLOGY

- **Root:** Topmost element.
- **Parent(i) / Children:** parent = `(i-1)/2`, left = `2i+1`, right = `2i+2`
- **Height / Leaf Node / Heapify:** Height is longest path to leaf. Heapify restores heap property.

8. KEY POINTS TO REMEMBER

- ✓ Heap is a Complete Binary Tree.
- ✓ Heap ≠ Sorted Array. Heaps are for Priority, not order.
- ✓ Choose Min/Max Heap based on the problem goal.
- ✓ Heap operations are logarithmic time (O(log n)).

9. COMMON MISTAKES

- ✗ Confusing Min Heap with Max Heap logic.
- ✗ Using wrong comparator with integer overflow risk.
- ✗ Forgetting that default PriorityQueue in Java is Min Heap.
- ✗ Not checking empty boundaries before poll() or peek().

HEAP PATTERNS Top Interview Patterns with Heaps

PATTERN 1: REPEATED MAX / MIN RETRIEVAL (e.g. Last Stone Weight)

When to Use?
Repeatedly extract max/min, perform arithmetic, and put back.


```java
PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
while(pq.size() > 1) {
    int diff = pq.poll() - pq.poll();
    if (diff > 0) pq.offer(diff);
}
```

PATTERN 2: TOP-K ELEMENTS (e.g. Kth Largest, Top K Frequent)

When to Use?
Find K largest/smallest. Keeps space bound to O(K).


```java
PriorityQueue<Integer> pq = new PriorityQueue<>();
for(int x : nums) {
    pq.offer(x);
    if (pq.size() > k) pq.poll();
}
```

PATTERN 3: MERGE K SORTED LISTS / ARRAYS (e.g. Merge K Sorted Lists)

When to Use?
Merge K sorted inputs efficiently. Seed heap with first node of each list.

Min Heap of size K

```java
PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b)->a.val-b.val);
while(!pq.isEmpty()) {
    ListNode curr = pq.poll(); tail.next = curr; tail = tail.next;
    if (curr.next != null) pq.offer(curr.next);
}
```

PATTERN 4: TWO HEAPS (MEDIAN FROM DATA STREAM)

When to Use?
Dynamic median lookup. Balance small (Max Heap) and large (Min Heap) halves.

Max (Small Half)
Min (Large Half)

```java
void addNum(int num) {
    small.offer(num);
    large.offer(small.poll());
    if (small.size() < large.size()) small.offer(large.poll());
}
```

Triggers: Top K, Kth Largest, Merge K Sorted Lists, Running Median, Task Scheduling. Next Page: Heap Decision Tree & Invariants

HEAP — CHEAT SHEET & DECISION GUIDE

FAANG Cheat Sheet — Decision Flowchart & Checklist PAGE 3

1. Heap Decision Tree Flowchart

2. Invariants & Checklist

Checklist:

- Ask yourself: What should I keep, and what should I discard?
- Decide Heap configuration based on sorting criteria (Ascending/Descending).
- Always check for extreme edge cases (k = 0, k > N).

3. AHA Moments

- Median = Balance two halves using Two Heaps.
- Heaps give you the optimal element in **O(1)** root access.
- Always check boundaries before calling `poll()`.

Invariants: Min Heap parent ≤ child; Max Heap parent ≥ child. Next Page: FAANG Hidden Triggers (Recognition Guide)

HEAP — FAANG HIDDEN TRIGGERS

How Interviewers Hide Heap Problems (Recognition Guide) PAGE 4

1. When to Think Heap? (Unseen Question Clues)

Question Says...	Think Heap Because...
"Top K / K largest / K smallest"	We only care about keeping the best K elements.
"K closest / K farthest"	Keep best K based on calculated distance.
"Kth largest / Kth smallest"	Heap retrieves Kth element in optimal time.
"Most frequent / Top frequent"	Use Heap on the frequency map key-value pairs.
"Continuously receive data"	Need to maintain running minimum/maximum.
"Running median / Median at any time"	Two Heaps partition the data in half efficiently.

2. Interviewer Tricks to Watch Out

- They never say "Use Heap" explicitly; they use priority terms.
- They mix multiple concepts (e.g., HashMap frequency + Heap ranking).
- They ask for both min and max simultaneously → think **Two Heaps**.

3. Heap vs Other DS

Goal	Best Tool
Get Min/Max repeatedly	Heap
FIFO order	Queue
Range Query	Segment Tree

GOLDEN RULE: Translate the problem into "Which element should come out first?" to determine priority.

POTENTIAL ISSUES FOUND

FAANG Cheat Sheet Validation & Quality Report PAGE 5

Technical Audit Findings

The following technical feedback lists potential issues or risks detected in the original Heap study notes. As per instructions, the original document content above has been recreated **exactly** as written without edits, and these notes serve as an external reference sheet for your study sessions:

Page	Original Text / Code	Suggested Correction	Reason	Confidence
Page 1	`maxHeap = new PriorityQueue<>((a, b) -> b - a);`	`maxHeap = new PriorityQueue<>((a, b) -> Integer.compare(b, a));`	Subtraction can trigger Integer overflow limits when values have opposite signs.	**High**
Page 2	Pattern 3 Comparator: `(a, b) -> a.val - b.val`	`(a, b) -> Integer.compare(a.val, b.val)`	Same subtraction overflow risk if node value differences exceed `Integer.MAX_VALUE`.	**High**
Page 2	Pattern 4 Median: `(maxHeap.peek() + minHeap.peek()) / 2.0`	`((double) maxHeap.peek() + minHeap.peek()) / 2.0`	If both integers are large, their sum will overflow prior to floating division.	**High**
Page 4	Missing cost warning for arbitrary deletes: `pq.remove(val)`	Add warning: `pq.remove(val)` is an **O(N)** operation.	Candidates often assume all Queue methods are O(log N) or O(1). remove(obj) is linear.	**High**